

Lecture 18:

Machine Learning Evaluation cont.

CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

Announcements

- PA5, WA5 out tomorrow, due next Sunday April 12

Model Evaluation cont.

What about confidence?

Confidence is how “sure” a model is about its predictions.

- Most ML algorithms can tell you how confident they are in an answer
- For example, the model can be 51% “confident” the point belongs to the positive class, or 99% “confident.”
- Confidence may be measured differently: for example, for KNN, if a out of k neighbors are Positive, the model is a/k confident the point is positive
- Confidence of $p\%$ does not mean that it's $p\%$ likely the classification is right!
- For *calibrated* models, it does mean that. We're not covering that.

What about confidence?

- In binary classification, $> 50\%$ confident in Class 1 usually means predicting Class 1.



Connecting this to evaluation

Let's say we want to measure the goodness of a single prediction – so, not say just “True Positive” – if we have access to a confidence metric for each prediction.

- If the model is 99% confident, and its prediction is correct, the error is close to 0.
- If the model is 51% confident, and its prediction is correct, the error is higher than 0.
- If the model is 51% confident, and its prediction is INCORRECT, the error is higher than that.
- If the model is 99% confident, and its prediction is INCORRECT, the error is very high.

Log Loss

The formula for a single sample is:

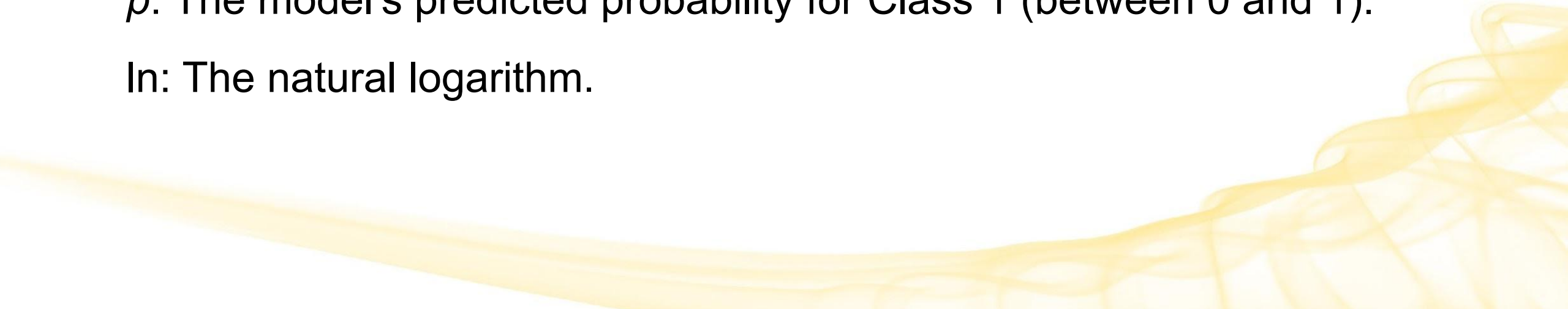
$$\text{Loss} = -(y \cdot \ln(p) + (1 - y) \cdot \ln(1 - p))$$

Breaking down the variables:

y : The true label (either 0 or 1).

p : The model's predicted probability for Class 1 (between 0 and 1).

$\ln$: The natural logarithm.



Log Loss

$$\text{Loss} = -(y \cdot \ln(p) + (1 - y) \cdot \ln(1 - p))$$

Case 1: The actual label is $y = 1$

The second half of the formula $(1 - y)$ becomes zero. You are left with:

$$\text{Loss} = -\ln(p)$$

If $p = 0.99$, the loss is $-\ln(0.99) \sim 0.01$ (Tiny penalty).

If $p = 0.01$, the loss is $-\ln(0.01) \sim 4.60$ (Huge penalty).

Log Loss

$$\text{Loss} = -(y \cdot \ln(p) + (1 - y) \cdot \ln(1 - p))$$

Case 2: The actual label is $y = 0$

The first half of the formula (y) becomes zero. You are left with:

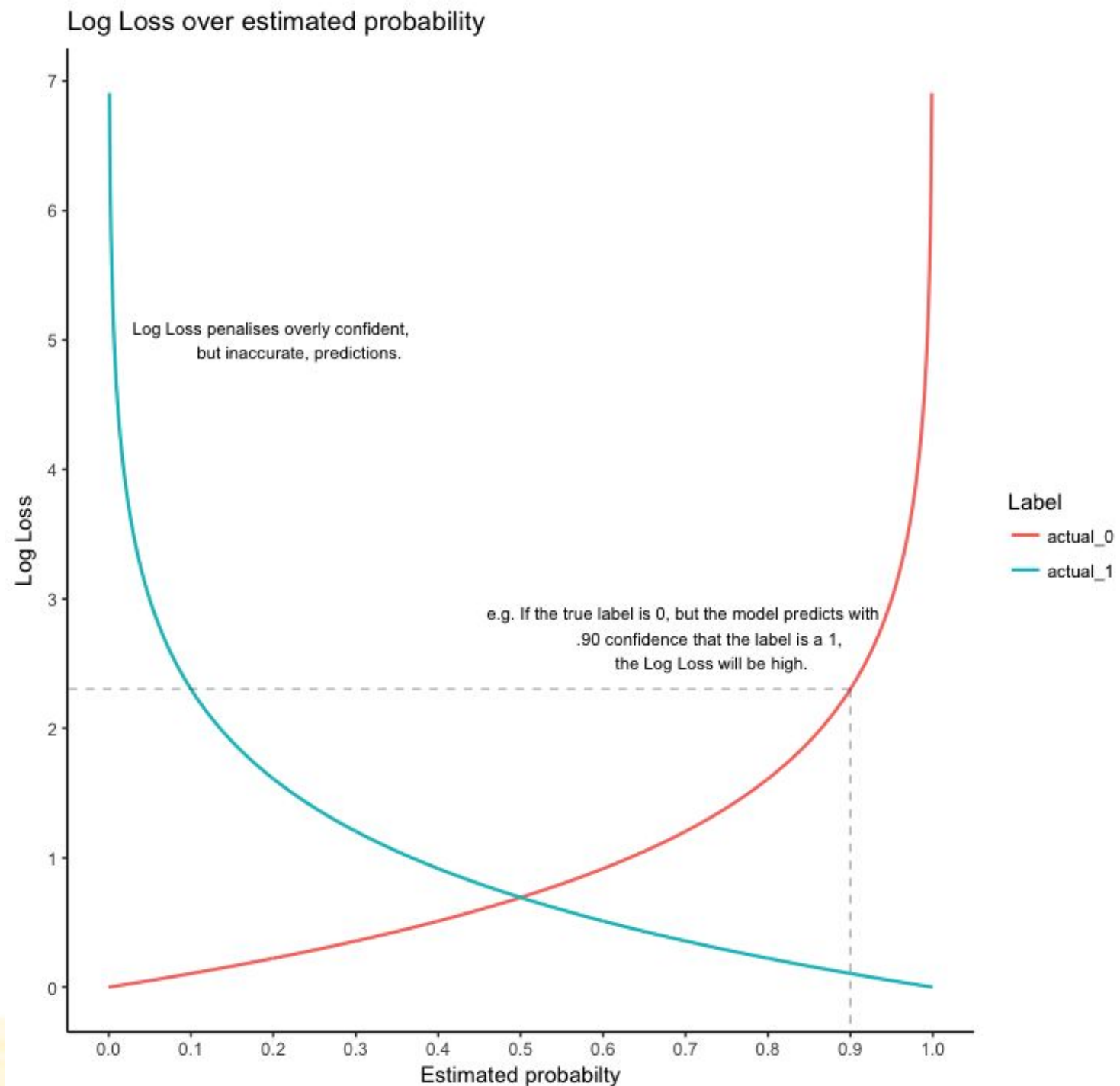
$$\text{Loss} = -\ln(1 - p)$$

If $p = 0.01$, the loss is $-\ln(0.99) \sim 0.01$ (Tiny penalty).

If $p = 0.99$, the loss is $-\ln(0.01) \sim 4.60$ (Huge penalty).

Log Loss

- Penalty increases as prediction probability diverges from actual
- The steeper the curve, the higher the cost of getting it wrong
 - A penalty for the model being “too confident” in a wrong answer, or “not confident enough” in a right answer



What if Confidence Matters?

Log Loss measures how accurate a model's predictions are, especially its confidence in those predictions.

Log loss: **A measure of error that penalizes overconfidence.**

- N = total number of data points
- l_i = individual losses

$$J = \frac{1}{N} \sum_{i=1}^N l_i$$

A lower log loss indicates better model performance, with values closer to 0 being ideal.

Log loss helps us understand how well the model's confidence matches reality.

Why LOG Loss?

The log function is used because it is asymmetric.

- Moving from 90% confidence to 95% confidence reduces your error a little bit.
 - Moving from 5% confidence to 0% confidence (when the answer was actually 1) increases your error toward infinity.
- 

Confidence example: multiple classes

Suppose we are classifying animals as 🐱, 🐶, or 🐭.

- Our model outputs a vector of how likely a given observation is to be each one of these:
 - $[p(\text{🐱}), p(\text{🐶}), p(\text{🐭})]$

Now let's say it gets an example that is 🐱.

- A perfect model would output $[1, 0, 0]$
- But our model isn't perfect (they never are)
- So instead we get $[0.8, 0.19, 0.01]$
 - 80% likely that it is a 🐱
 - But still 20% it is not
- Technically the answer is correct, as the model will produce label 🐱
- But it's not as good as a $[1, 0, 0]$ output!

Log Loss extension to multiple classes

When you move from binary (2 classes) to multiclass (3+ classes): we punish the model based on the probability it assigned to the correct (ground truth) class.

This generalization does actually apply to the case of 2 classes, too.

If interested, confirm arithmetically that calculating p as the probability assigned to the correct class will lead to the same result as calculating it as the probability assigned to class 1 (which is what we said we do).



General Framework for ML Evaluation

You have data, and you want to model a pattern in it so that you can predict or classify new, unknown observations.

Two natural questions arise:

1. How well will the model perform on previously unseen examples?
2. Is there a better model than the one we are using now?

Our ability to answer these depends on the assumption of statistical inference:

- That our sample training data is representative of the wider population/data.

In other words

If you train solely on:



You cannot predict:



Which Metrics of Performance in ML?

Which metrics to use depends on the type of algorithm/problem at hand.

- **Regression:**

- Usually Mean squared error (MSE) (*future topic*)
 - Sometimes Root Mean Squared Error (RMSE)
- Measures how close a regression line is to a set of data points.

- **Classification:**

- Depends
- ex. Binary/2-class classification:
 - Confusion Matrix and related measures (precision, recall, accuracy, etc)

- **Clustering:**

- A whole other class of metrics we haven't talked about yet! (*future topic*)
- ex. Silhouette Score

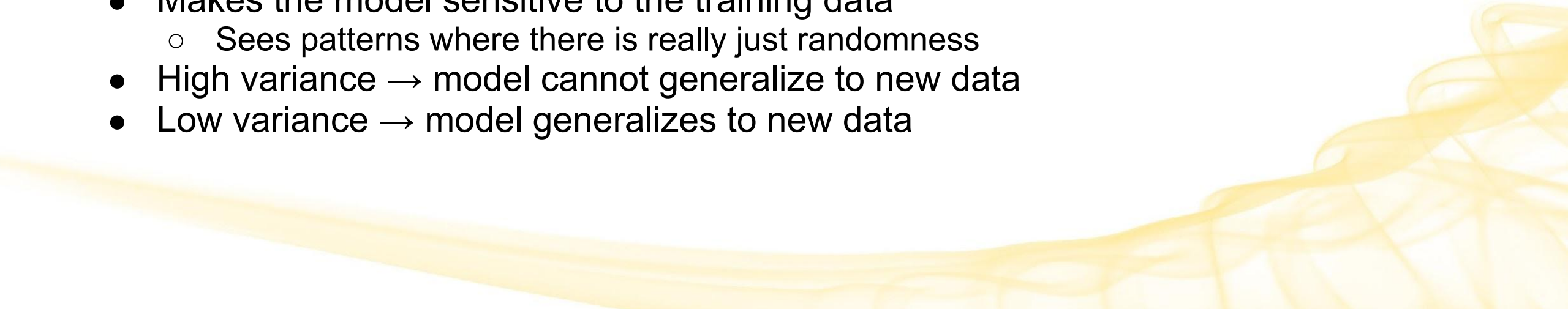
How good is our model?

ML pitfalls: Bias vs Variance

Bias

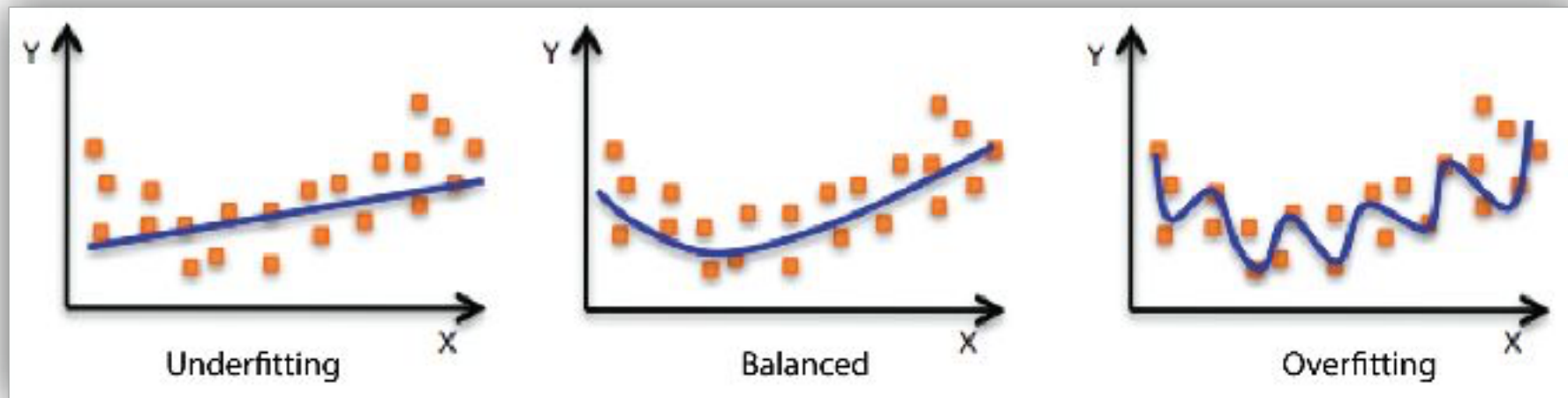
- Happens when the model is **too simple** (makes too many assumptions)
- Causes the model to overlook certain patterns/relationships in the data
- High Bias → model doesn't match the data set closely enough
- Low Bias → model matches the data closely

Variance (⚠ Note: different variance than statistical variance!)

- Happens when the model is **too complex**
 - Makes the model sensitive to the training data
 - Sees patterns where there is really just randomness
 - High variance → model cannot generalize to new data
 - Low variance → model generalizes to new data
- 

Things that affect model performance

Overfitting and **Underfitting** are two crucial concepts in machine learning and are the prevalent ***causes for the poor performance*** of a machine learning model.



Overfitting



Overfitting

Overfitting happens when the model becomes too tuned on the training set, and becomes unable to **generalize**.

Think of this as taking the same practice exam over and over and doing really well because you've already taken it. That doesn't necessarily make you better at taking a new, different exam.

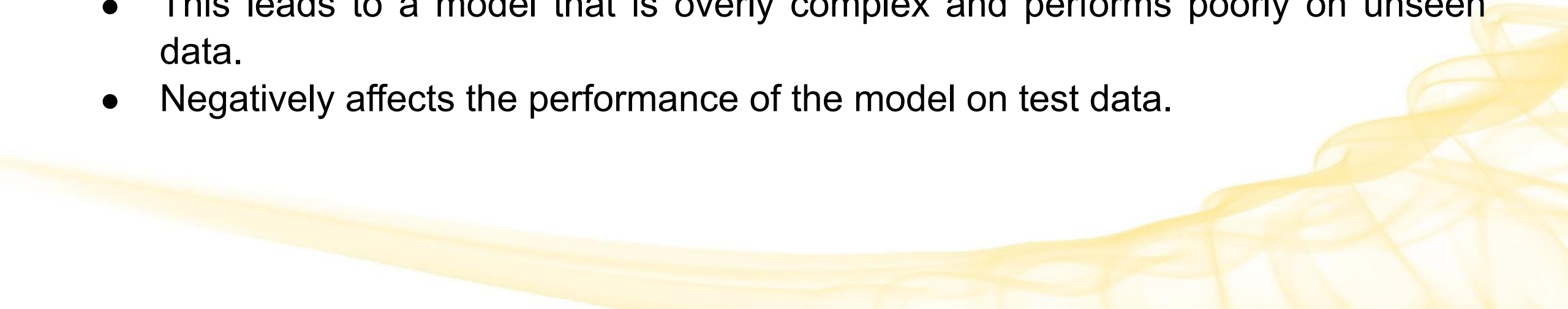
Overfitting is an **undesirable** machine learning behavior that occurs when the ML model gives **accurate predictions for training data but not for new unseen data**.

high variance and low bias

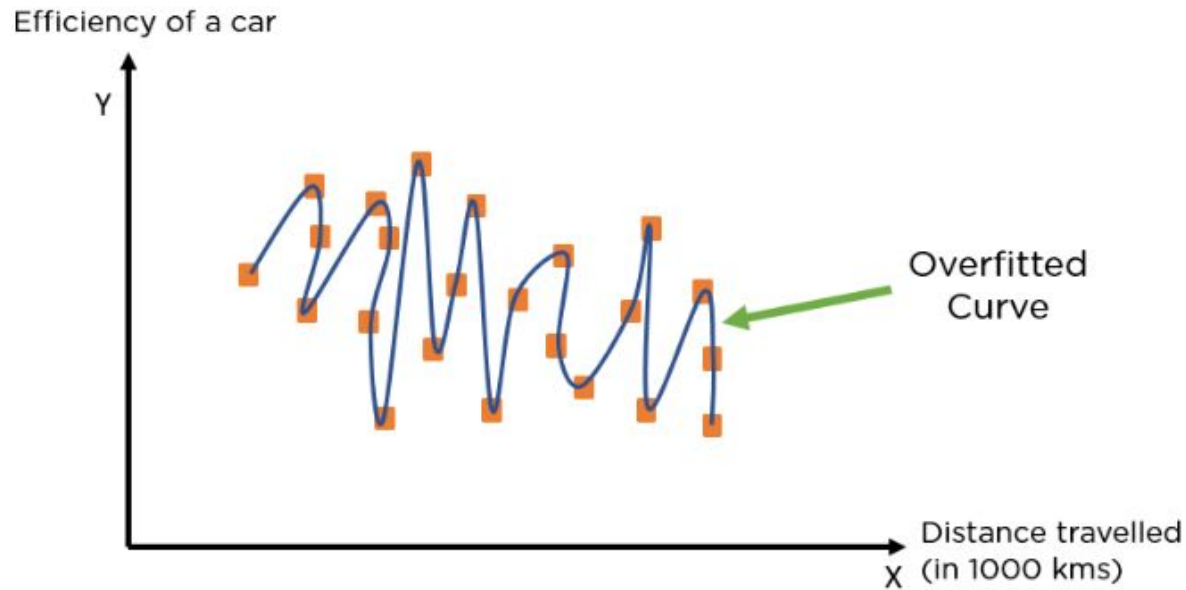
Overfitting

Definition: When a model performs very well for training data, but has poor performance with test data (new data).

How does it happen? The machine learning model **learns the details and noise in the training data too well**, capturing noise or random fluctuations in the data rather than the underlying patterns.

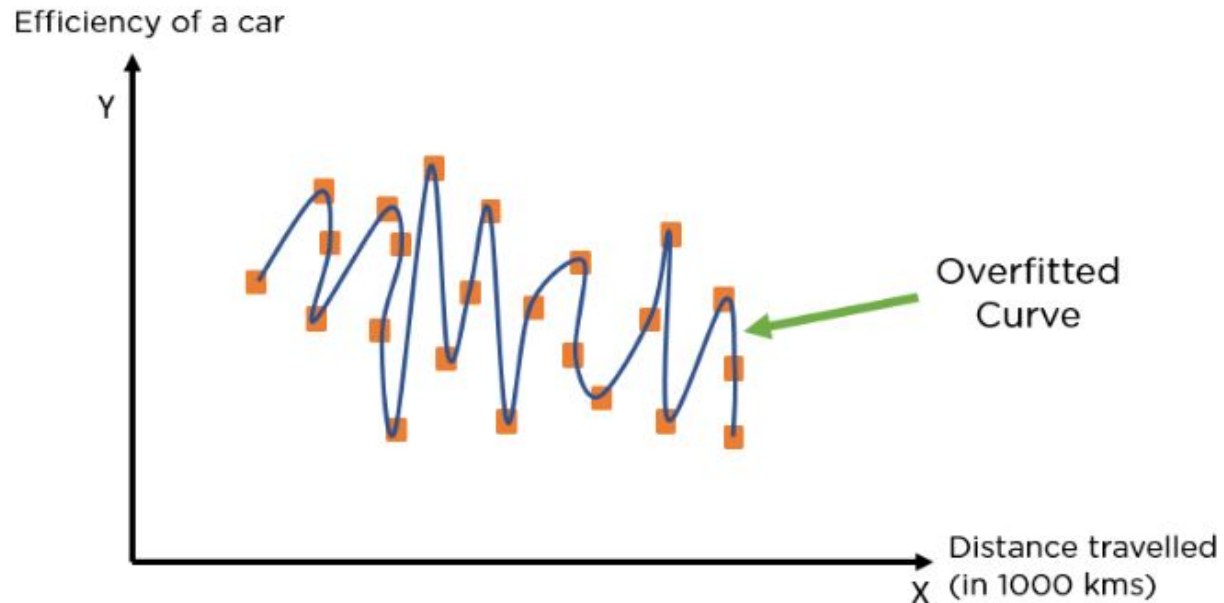
- This leads to a model that is overly complex and performs poorly on unseen data.
 - Negatively affects the performance of the model on test data.
- 

Reasons for Overfitting



- The **training data contains** large amounts of irrelevant information, called **noisy data**.
- The **model is too complex** (high variance, low bias): so it learns the noise within the training data.
- Instead of understanding the real patterns, it starts **memorizing the training data**, including noise and random fluctuations.
 - Hinders ability to generalize to new, unseen data

Reasons for Overfitting (cont.)



- The **size of the training dataset** used is **too small**
 - Too few examples: lacks diversity and leads to models memorizing noise instead of learning true patterns

How do you know that you overfit?

Your model performs well when you feed it training data, but then performs poorly when you feed it new, unseen data.



Some ways to Tackle Overfitting

- **Train the model with sufficient, quality data**
 - Add more diverse and representative data
 - Leads to more robust and generalizable models.
- **Reduce model complexity**
- **Use K-fold cross-validation** (*splitting train-validation-test – will discuss in a bit*)
 - Helps evaluate model performance and promote generalization to new data.
- **Use Regularization techniques** (*future topic*)
 - Reduce overfitting by adding penalties to model parameters, making them less sensitive to noise
 - Some of the same techniques(ex. LASSO, Ridge) can be used for the Embedded Method of Feature Selection in Feature Engineering
- **Ensemble methods** (*future topic*)
 - Combine multiple models to improve generalization
 - Ex. Bagging, boosting, stacking

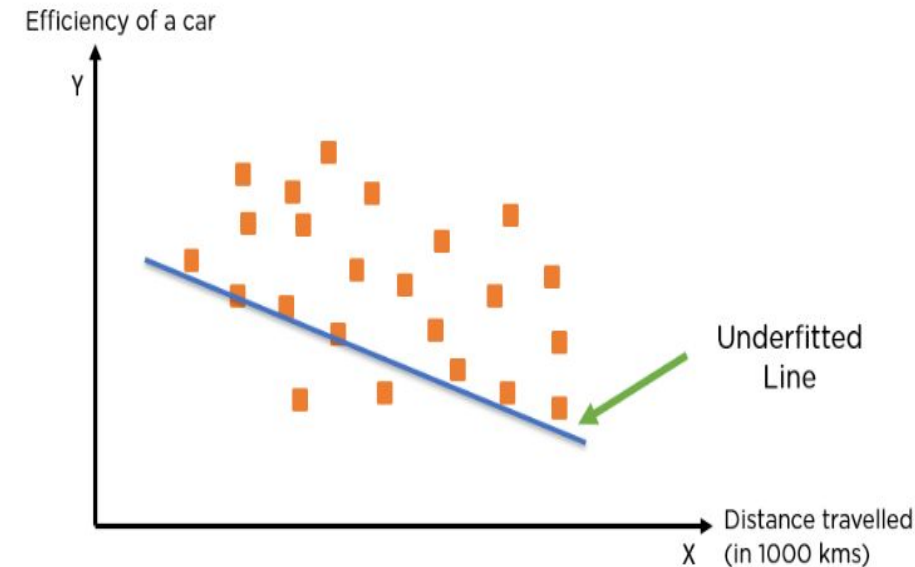
Underfitting



Underfitting

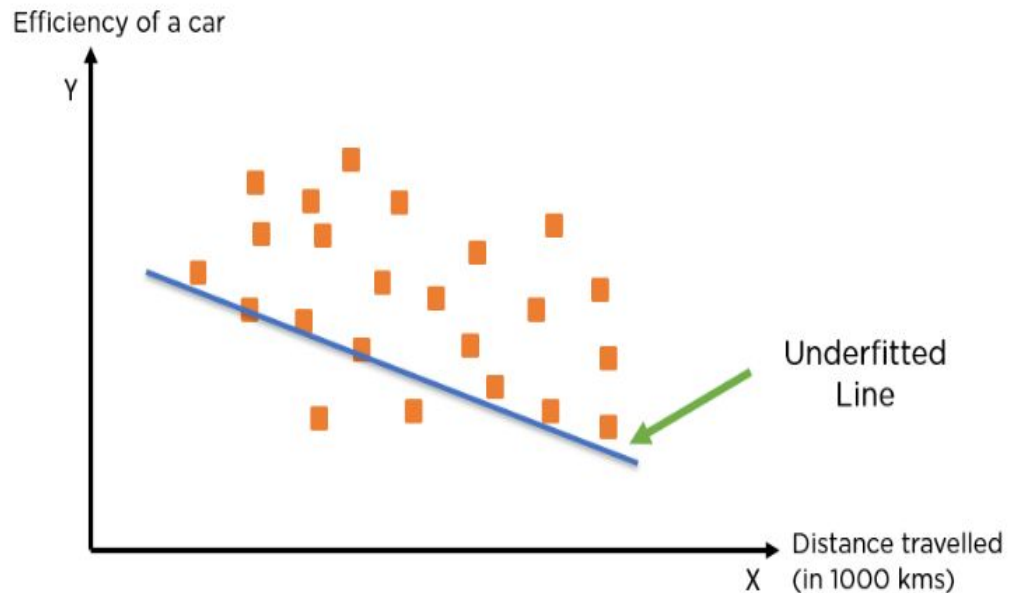
The model is **too simple/general**.

- The opposite of overfitting (sort of)
- The model isn't able to capture patterns/relationships in the training data correctly, and thus performs poorly on both training data and new, unseen examples.
- Examples:
 - Trying to fit a straight line to a polynomial relationship
 - Trying to fit everything you find to the same simple, learned rule ("shoe that fits anything")



low variance and high bias

Reasons for Underfitting

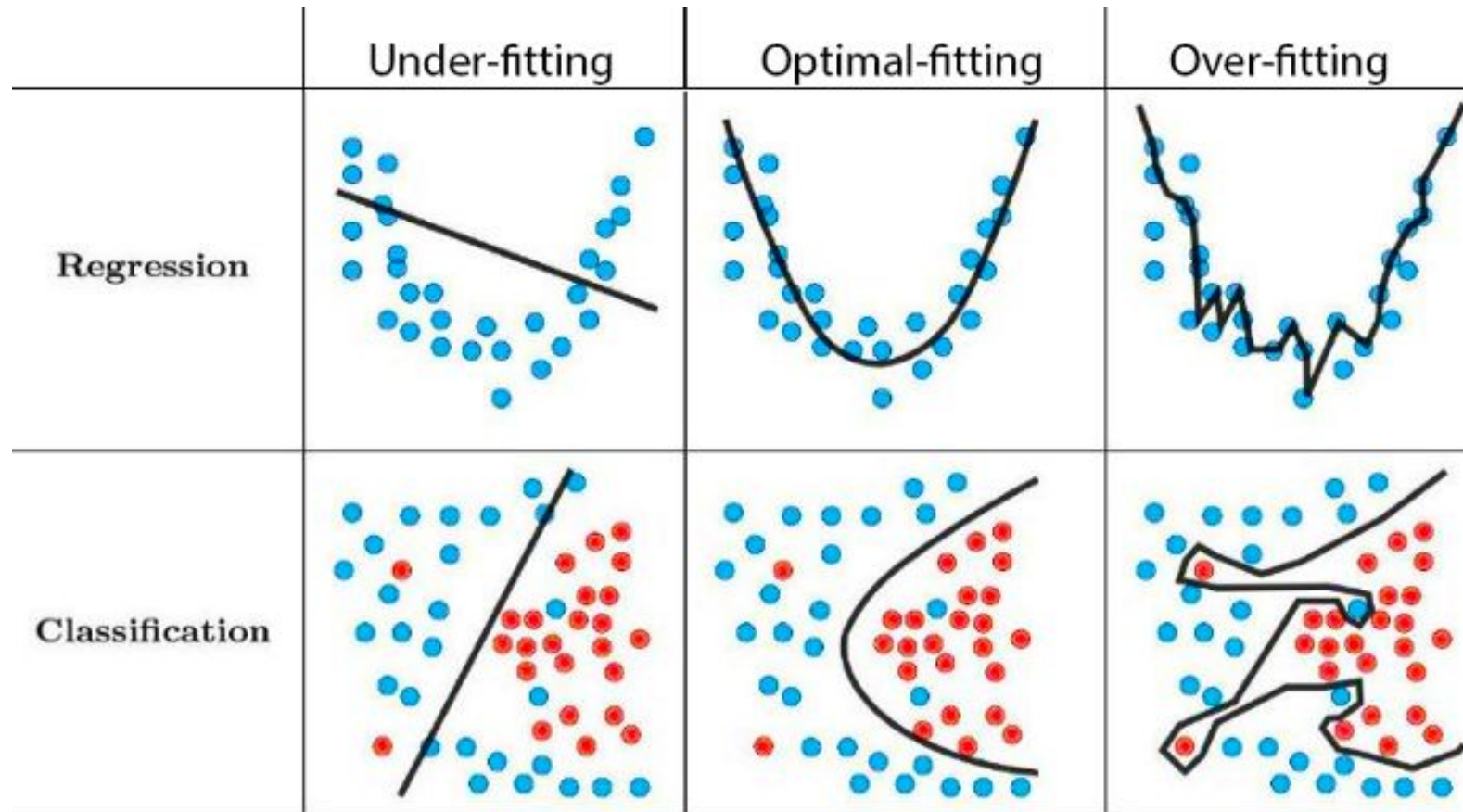


- **Not enough** training data
- **Not high enough quality** training data
 - Lacks variety needed to represent the population
 - Lacks the features needed to model
- The **model is too simple** (low variance, high bias), so it misses patterns/relationships in the data.

Some ways to Tackle Underfitting

- If the model is too simple to capture the complexity of the data, **increasing model complexity** can help alleviate that.
 - Examples:
 - Add More Layers or Neurons in Neural Network (*future topic*)
 - In polynomial regression, you can increase the degree of the polynomial to allow the model to fit more complex curves to the data.
- **Feature Engineering**
 - Add more relevant features and remove irrelevant or redundant ones
- **Get more/better data**
 - Get a sample that better reflects the population you want to model

Regression and Classification Examples



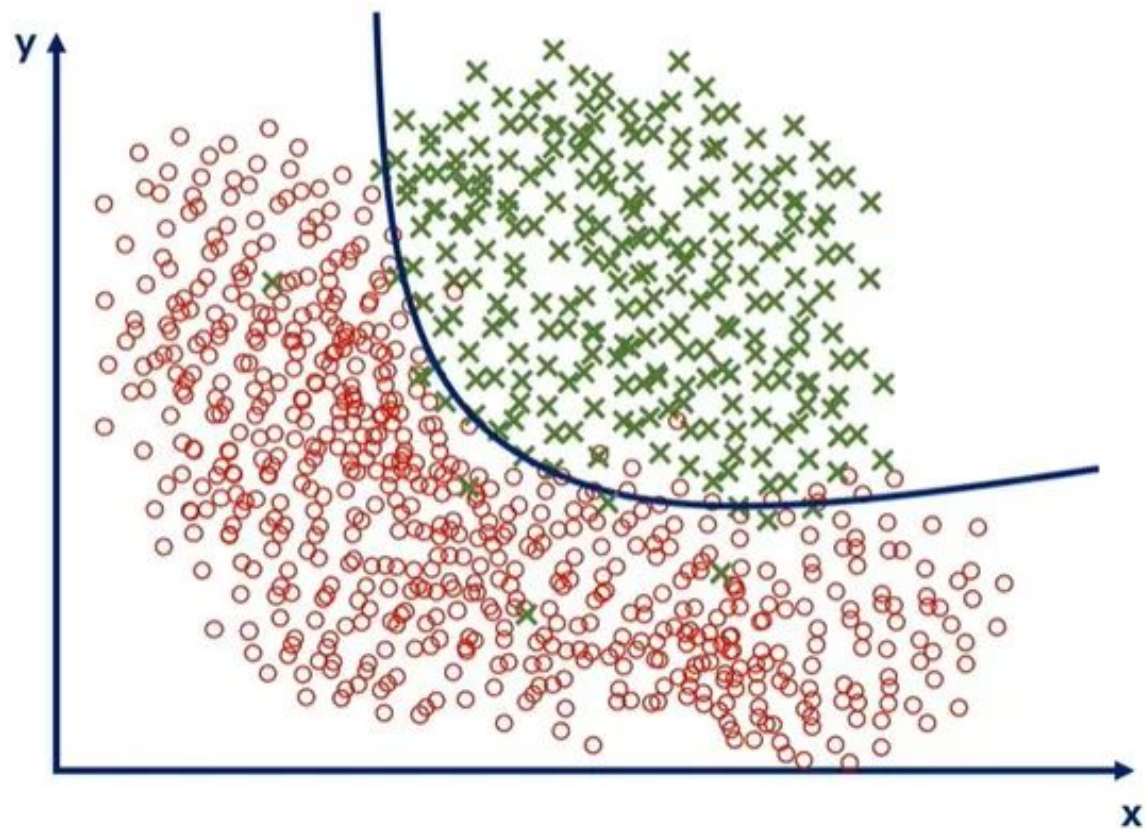
Overfitting vs Underfitting

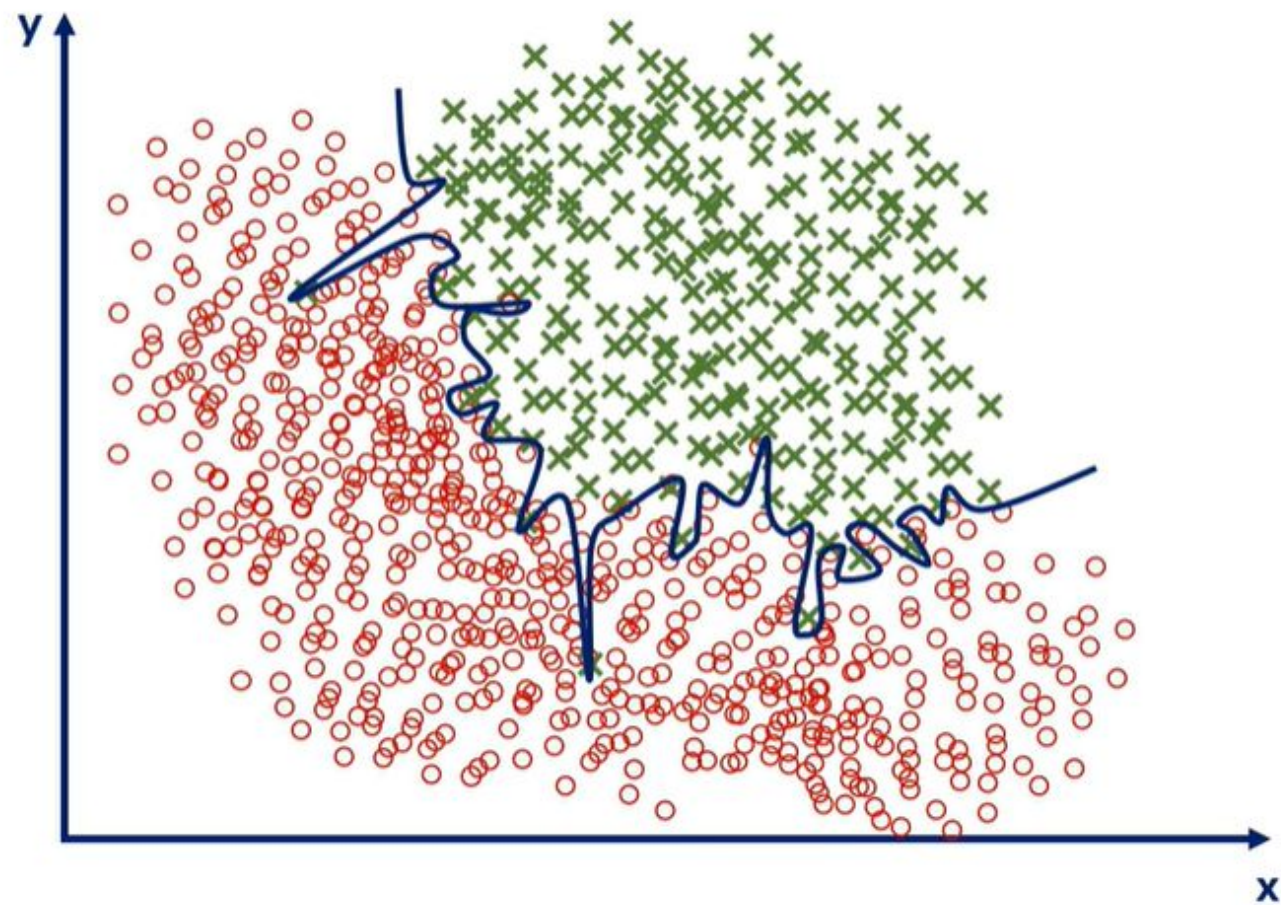
	Overfitting	Underfitting
Definition	Model is too complex (captures noise), fails to generalize.	Model is too simplistic, lacks generalization.
Bias vs. Variance	high variance, low bias	low variance, high bias
Causes	Too many parameters, limited data, excessive complexity.	Simple model, insufficient features, under-training.
Training Performance	Performs exceptionally well on training data.	Performs poorly on both training and new data.
Generalization	Poor generalization to new, unseen data (low performance).	Limited ability to capture underlying patterns (low performance).
Typical Solutions	Reduce model complexity, more training data, regularization.	Increase model complexity, feature engineering, more data.

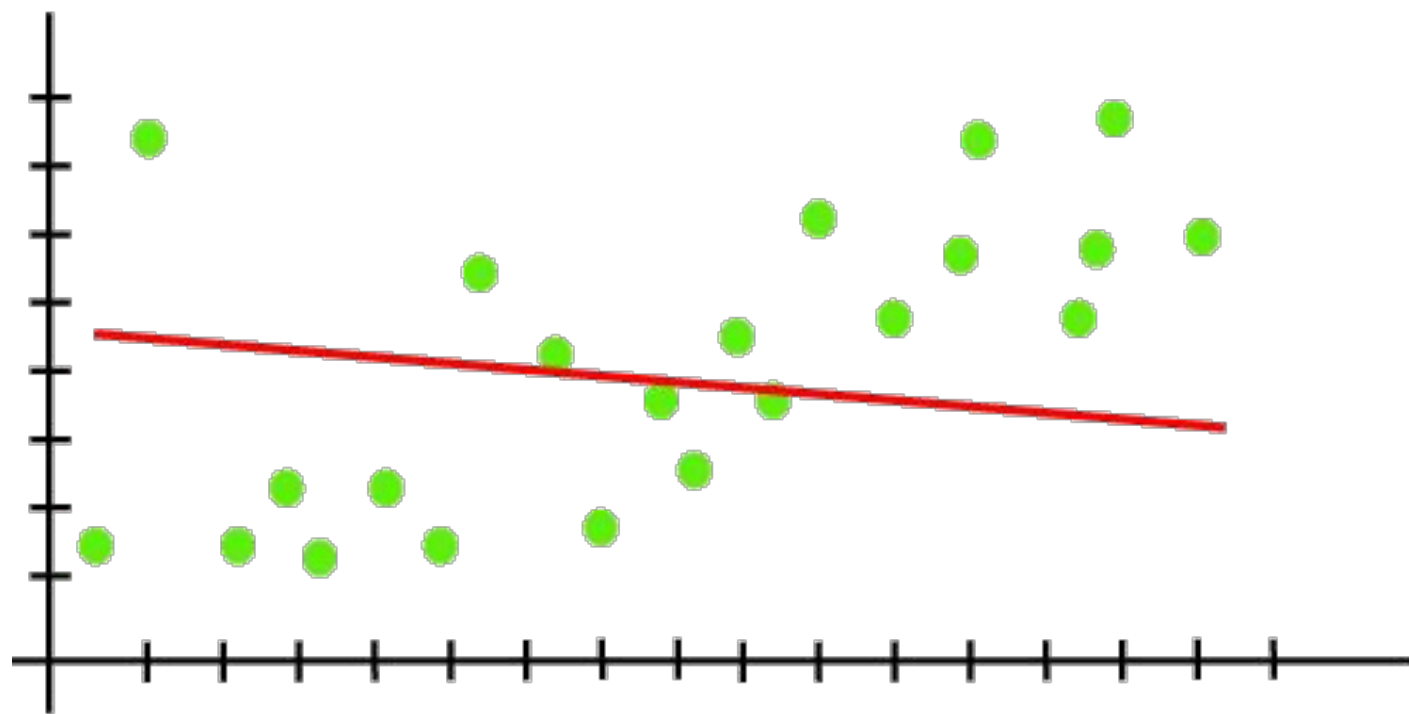


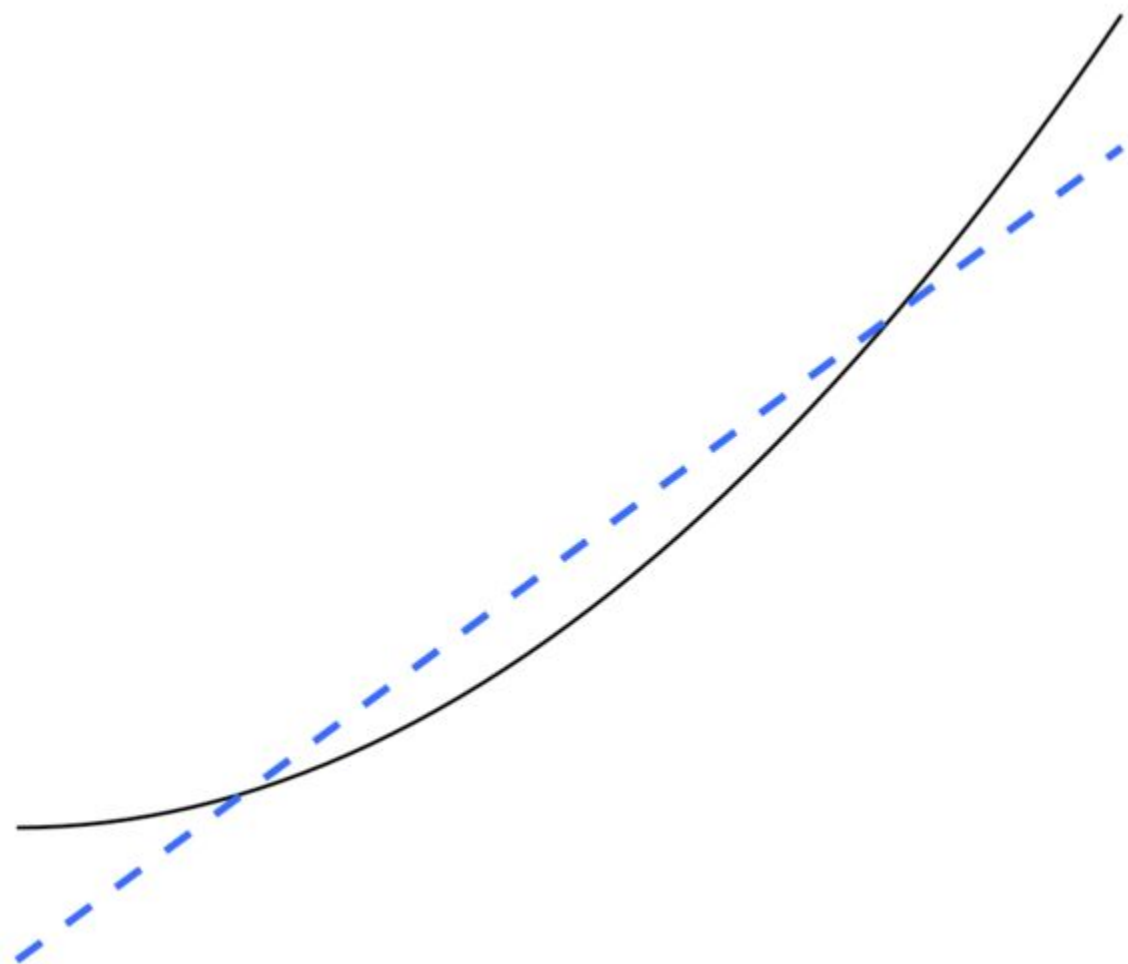
A Game:

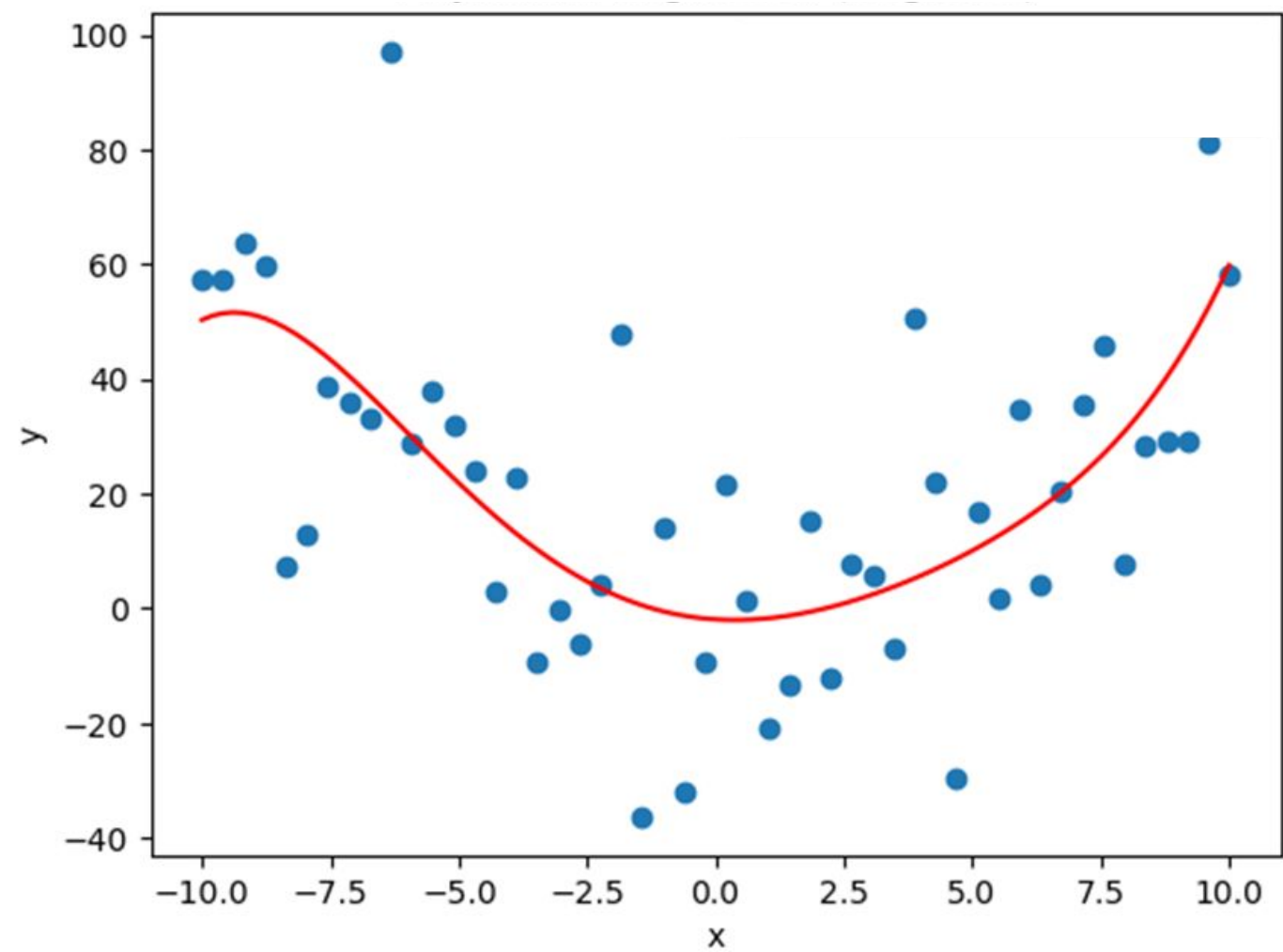
Is that Underfitted, Overfitted, or neither?

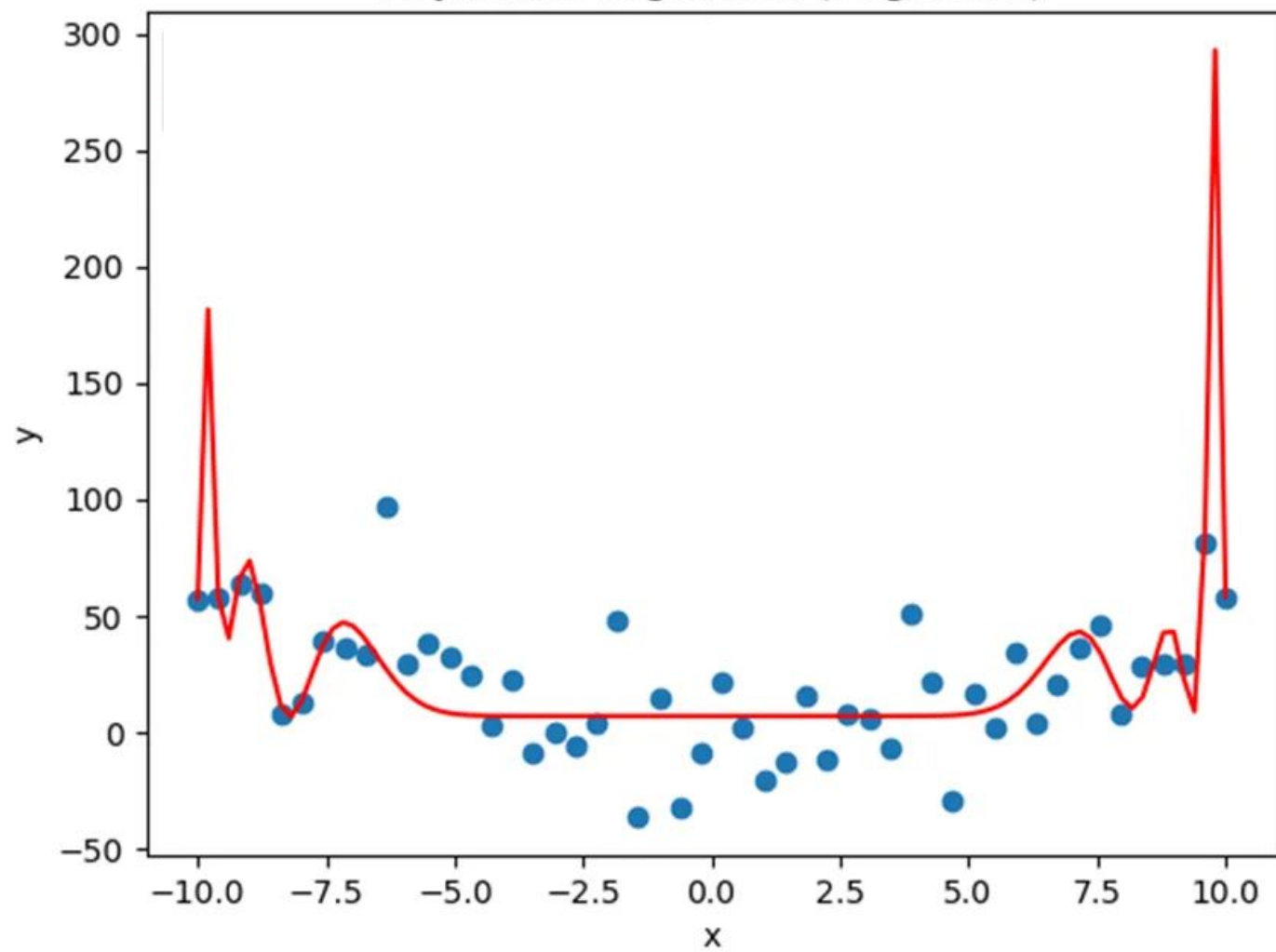










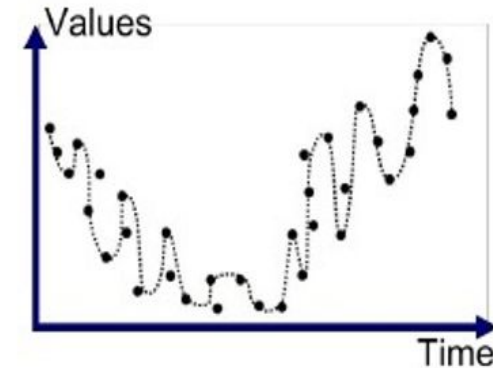
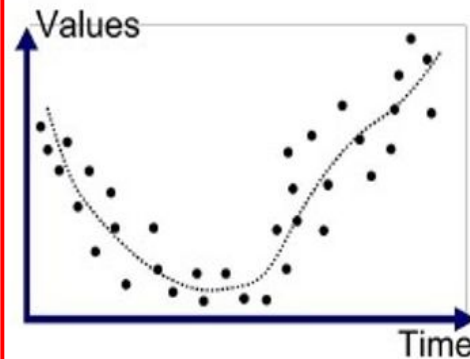
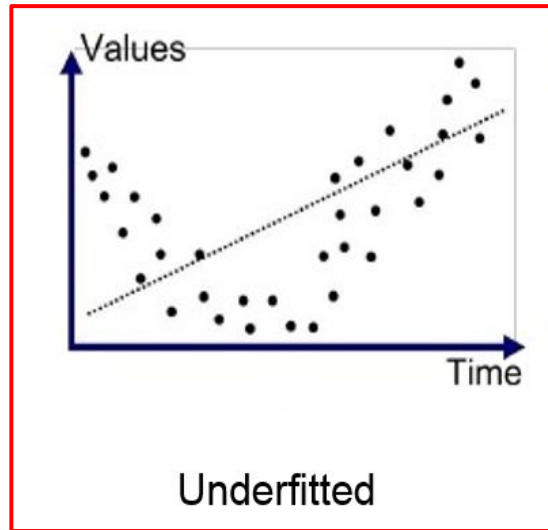


Bias/Variance Tradeoff



Another look at Bias

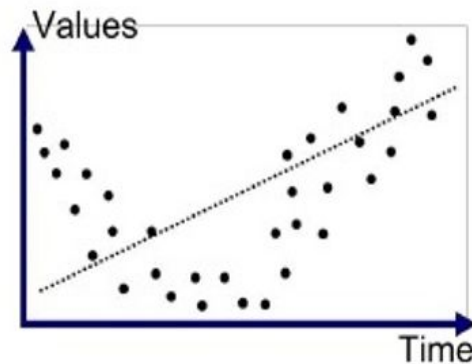
Bias refers to the error introduced by approximating a real-world problem (which may be complex) by an overly simplified model.



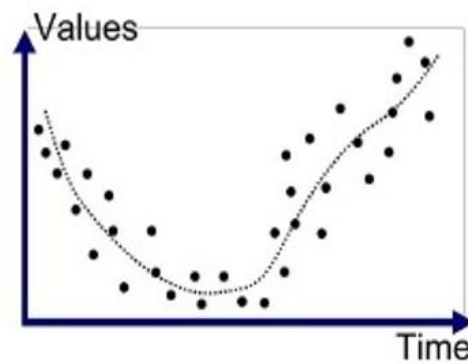
Bias quantifies how much the predicted values differ from the actual (expected) values.
High bias → simple model → poor performance.

Another look at Variance

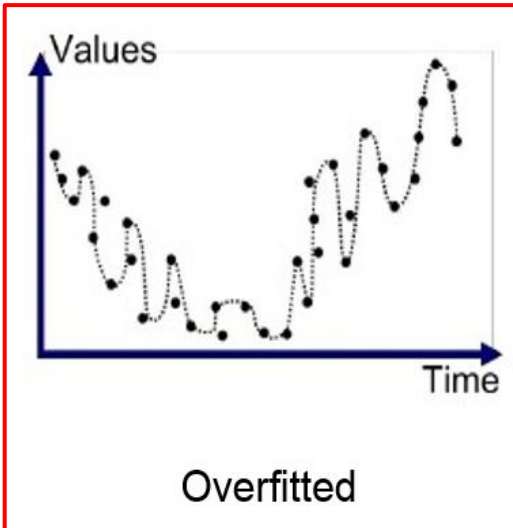
Variance is the error introduced when a model is *overly sensitive* to fluctuations in the training data (measures how much the predictions for a given data point can change when the model is trained on different subsets of the data.).



Underfitted



Good Fit/Robust



Overfitted

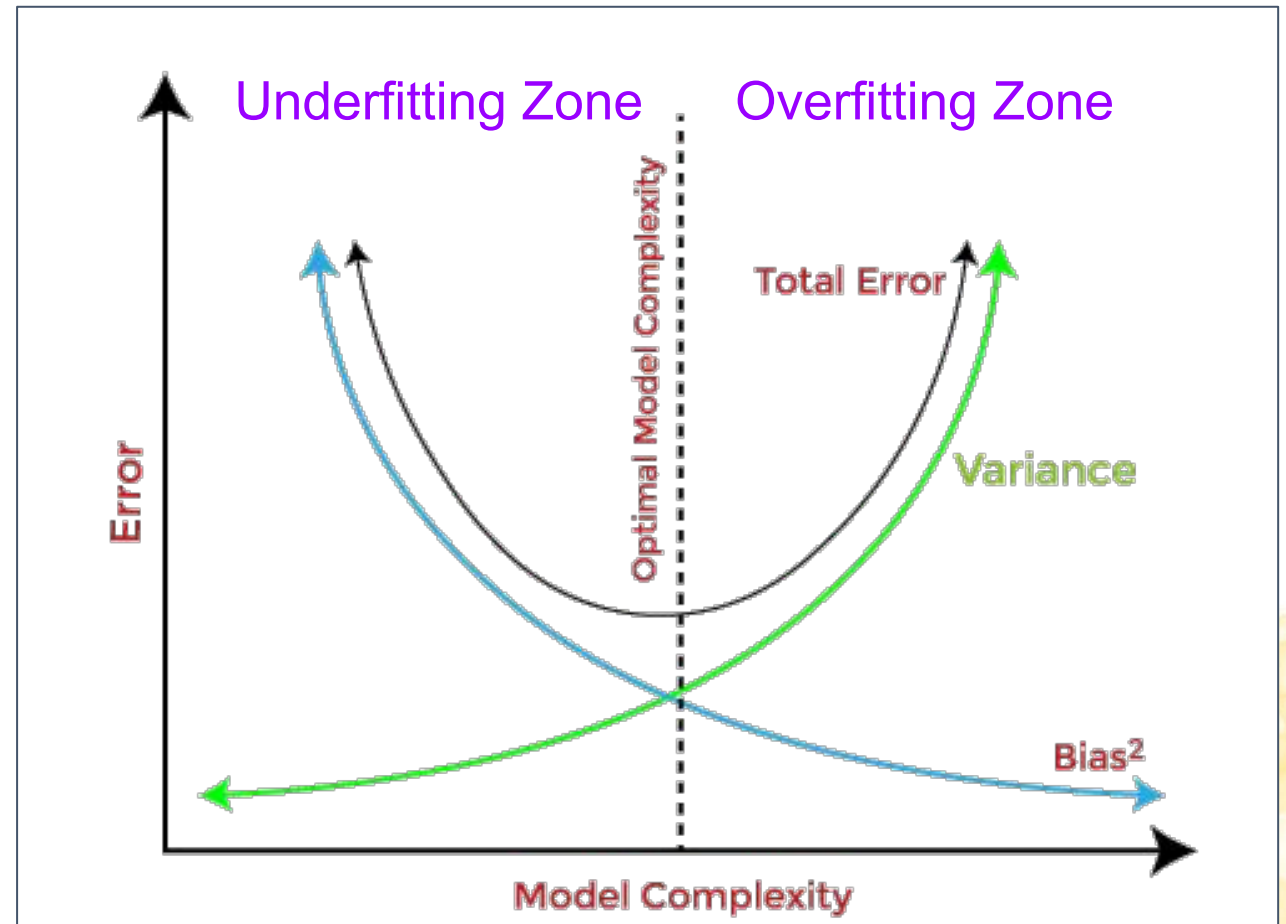
Variance is the variability of model prediction for a given data point or a value which tells us spread of our data.

High variance → complex model → great performance on training data but poor on new data.

Bias vs Variance Tradeoff

In ML, there is a relationship between model complexity and prediction accuracy for seen and unseen data.

The challenge: How do we balance simultaneously **minimizing bias error** and **variance error**? How do we avoid both overfitting and underfitting?

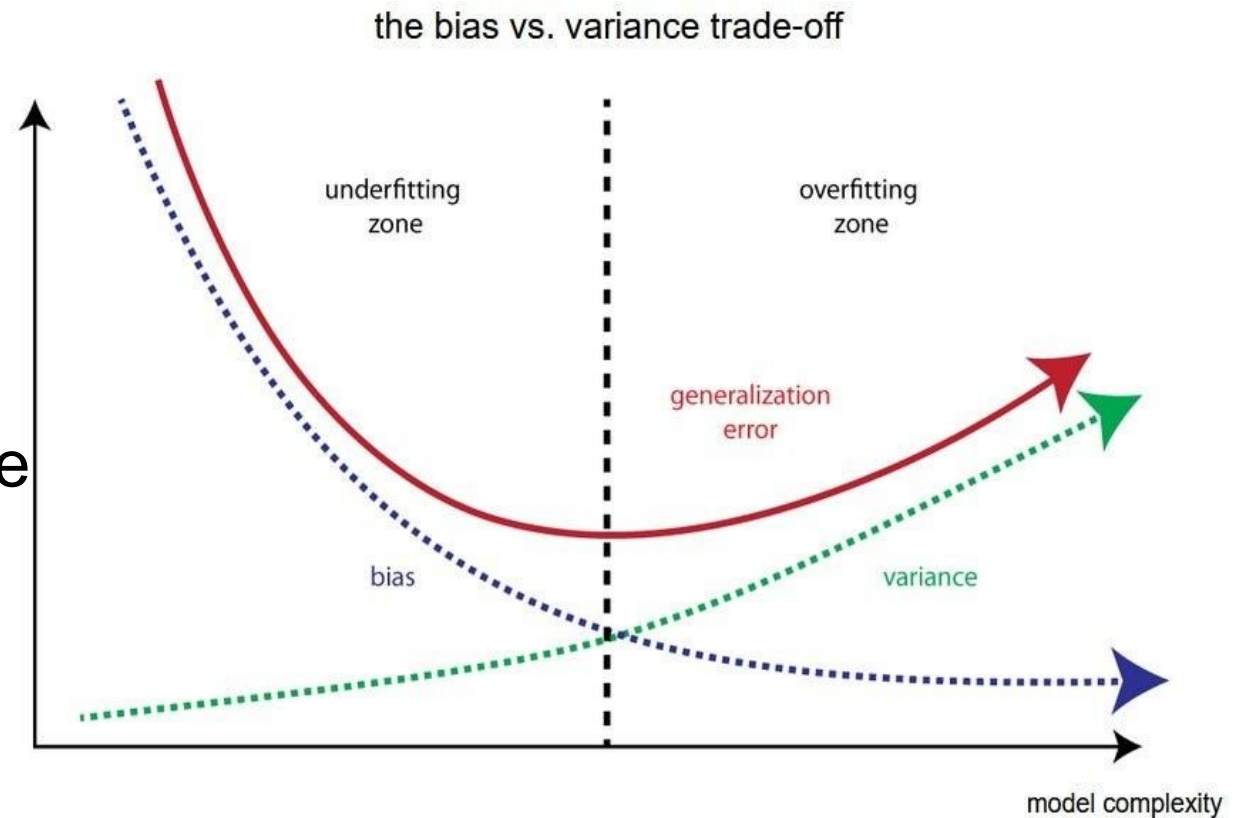


Bias Variance Decomposition

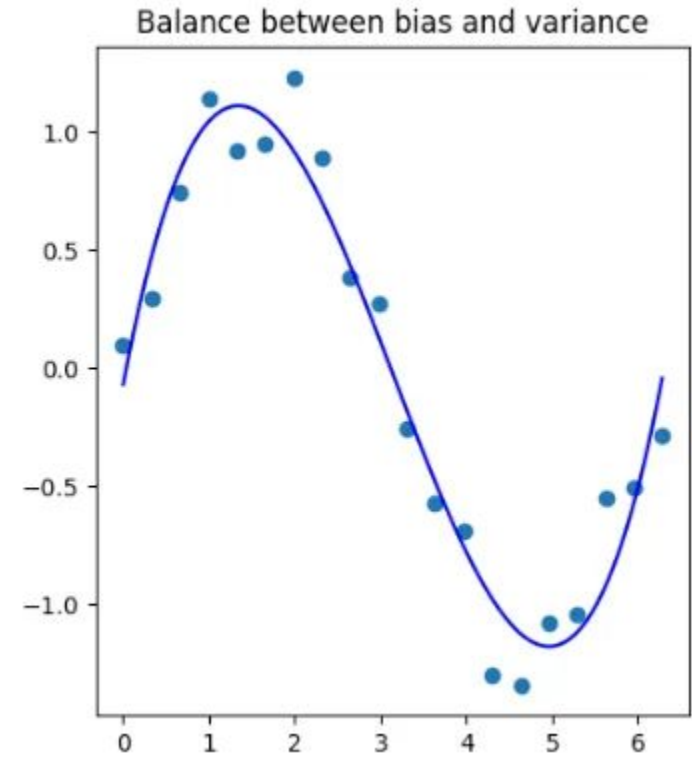
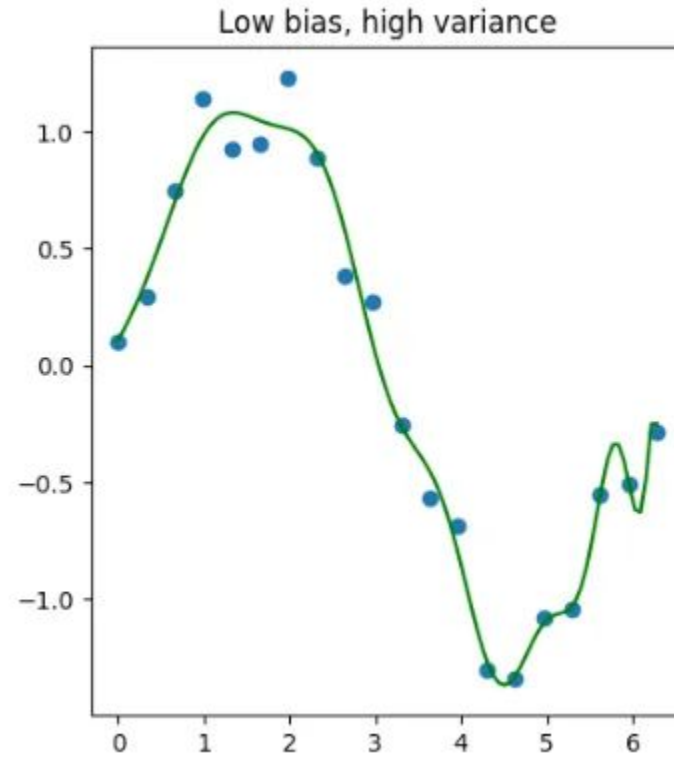
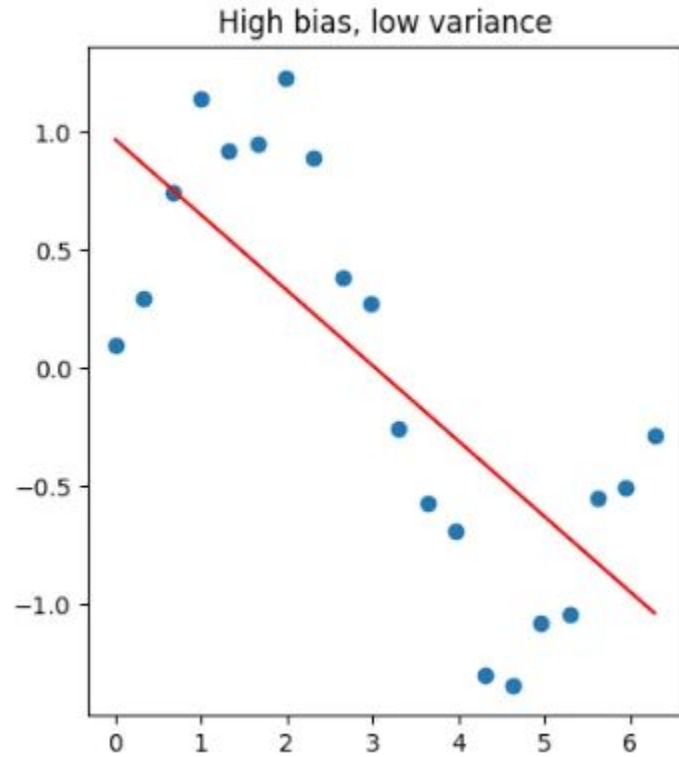
Aim for a model that minimizes both bias and variance for optimal performance.

Generalization Error (expected)
 $= \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$

Irreducible Error: Noise inherent in the data itself, which cannot be eliminated, regardless of the model.



Balancing Bias and Variance



Combating Training Failures



Validation



Often in Machine Learning Models

The point of having models is that, **once they are trained, they will be able to predict things about *new* data.**

In our bank loan example, what we're hoping is that the model uncovered the *underlying rules* about who repays loans and who doesn't.

- ex. Indicators like "higher income".
- 

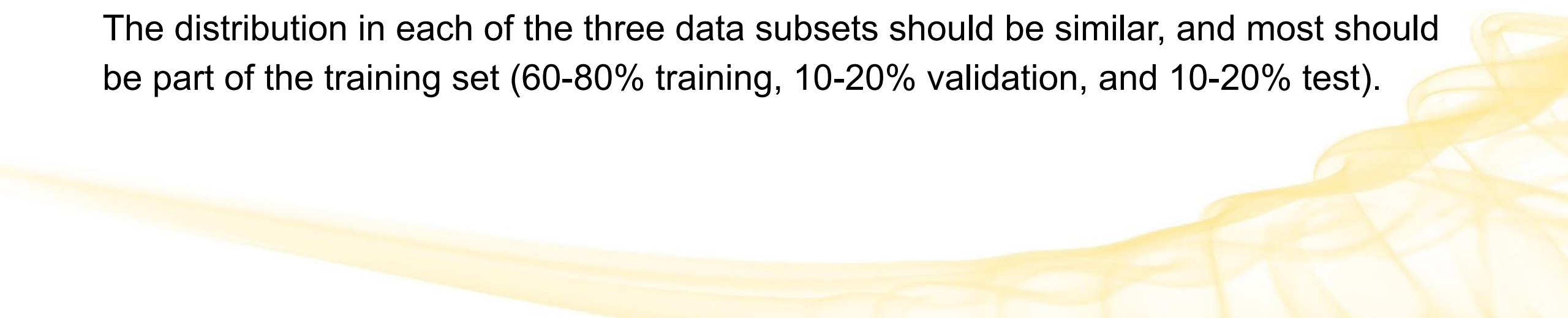
The Importance of Data Splitting in Model Training

Issue: Models trained on sample data carry the **risk of overfitting** to the sample data, leading to poor performance when predicting beyond the sample.

To compensate, sample data is often split into three subsets:

- **Training data** is used to **fit a model**.
- **Validation data** is used to **evaluate model performance while adjusting parameter estimates and conducting feature selection, or when comparing models**.
- **Test data** is used to **evaluate final model performance**.

The distribution in each of the three data subsets should be similar, and most should be part of the training set (60-80% training, 10-20% validation, and 10-20% test).



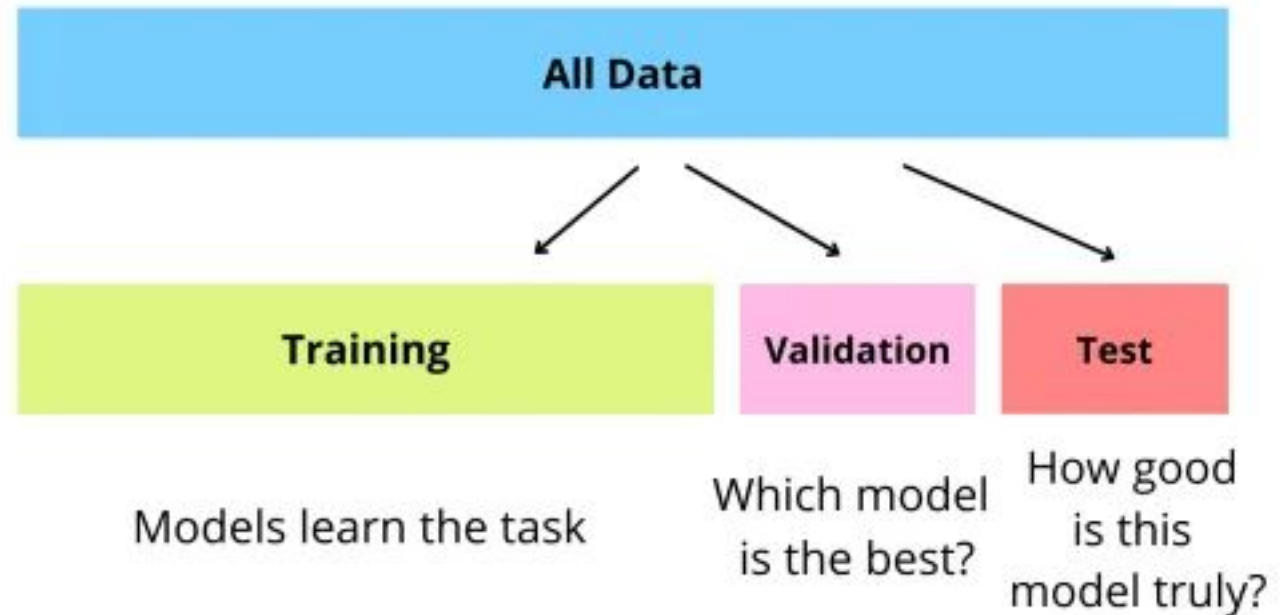
More Split: Train and Validation and Test

The train set → train the model

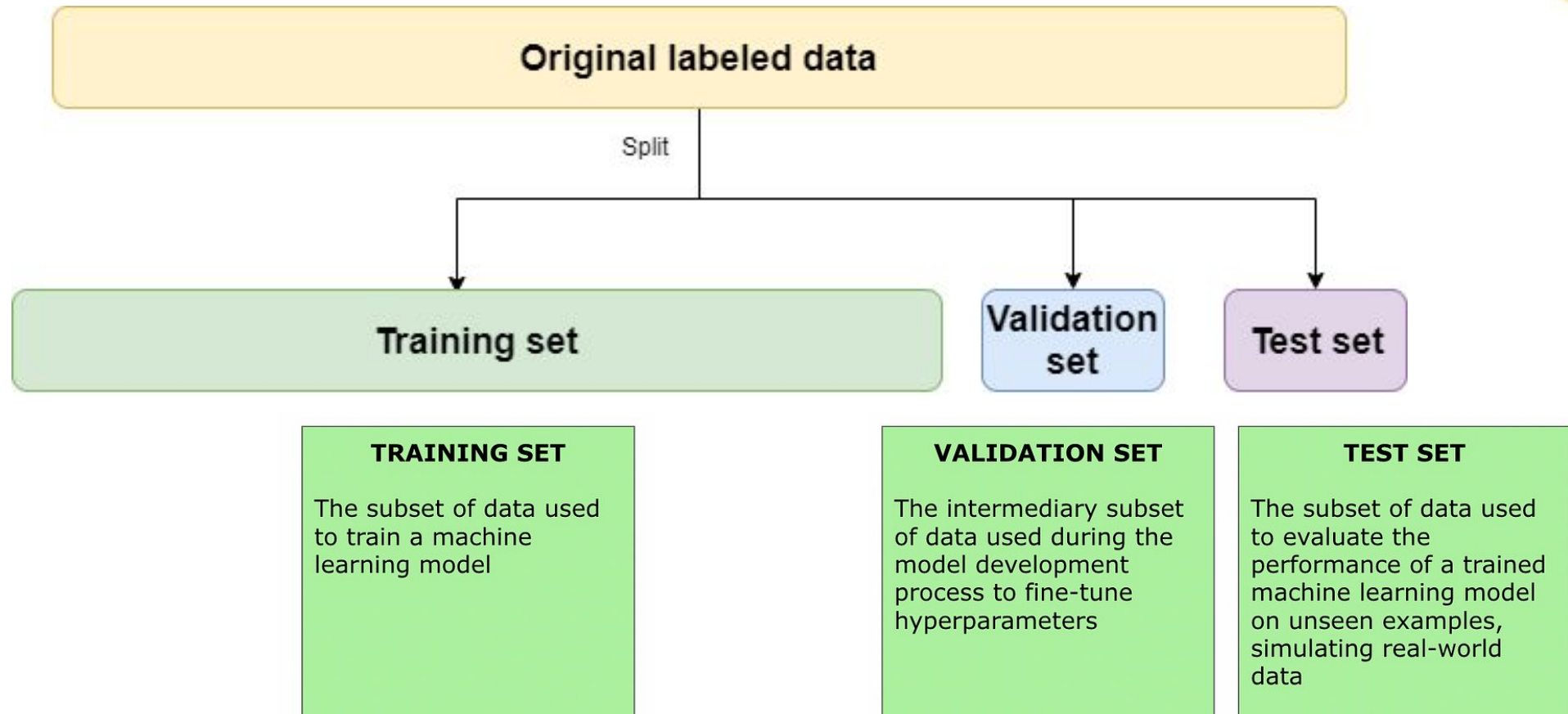
The validation set → tune the hyperparameters of the model to improve the performance

The test set → evaluate the model, on data that was not used for training

In machine learning, both the test and validation datasets are used to evaluate the performance of a trained model, but their purposes are different!

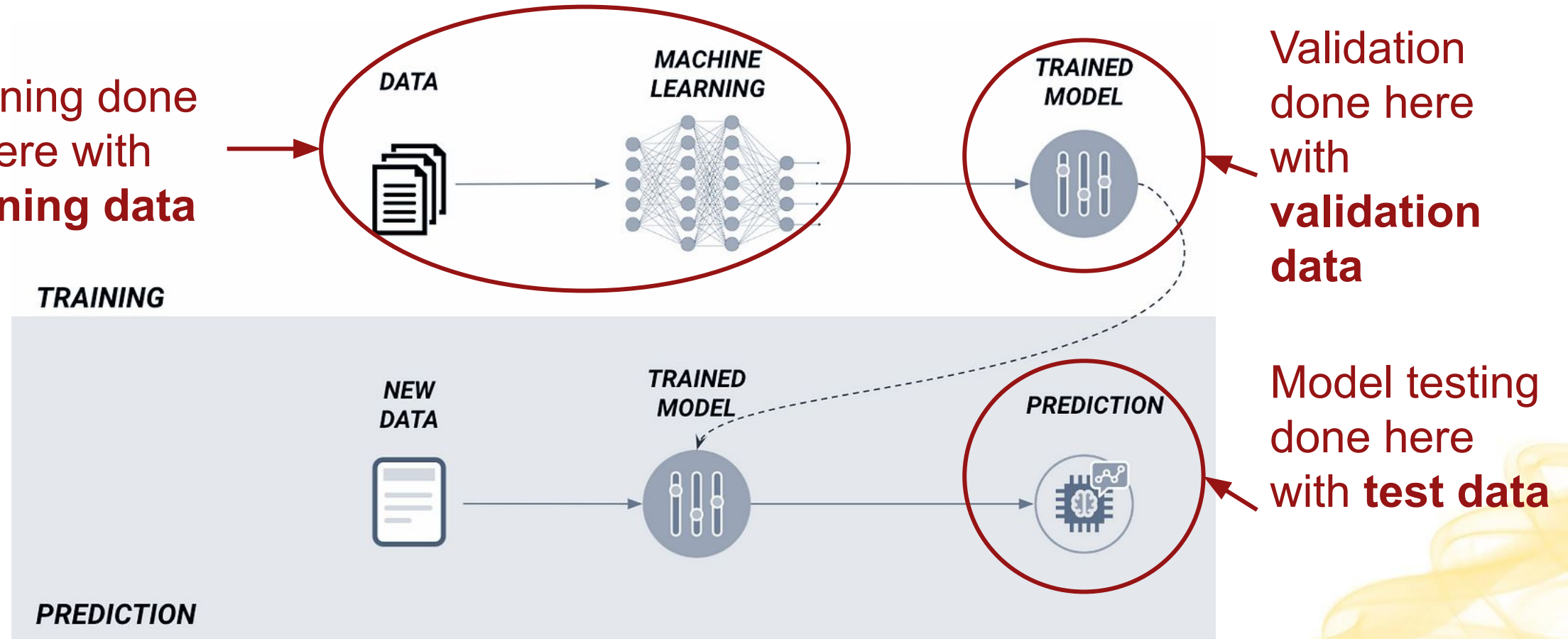


Dataset Splits



Again

Training done here with **training data**



Validation Sets

Validation data is a separate part of data, but is often taken from a subset of the training data and is used iteratively during model development.

- Helps evaluate the model's performance during the training phase.
- **The model is not trained on this data**
 - Instead, it validation data is used to assess how well the model generalizes to unseen examples and helps make decisions about fine-tuning (adjusting) hyperparameters or model architecture.

Test vs Validation Split

The **test dataset** helps us to **evaluate the model's performance on new data**.

The **validation set** helps us to **tune the hyperparameters of the model** so that it is not overfitting to the training data.

- Check if the model has learned training the data too well and is unable to generalize to new data from the validation set.



Ensuring Robust Model Evaluation

We need to be **careful about how we split our data** so that we don't unintentionally introduce an uneven distribution.

- Issues can arise if the split isn't random
 - Ex. one part of the data only includes people from a specific state, employees with a certain income level, or only women, it can make the model learn too much about these specific cases (and overfit)
 - Especially prone if the dataset is ordered on a feature we care about
- 

Demo

